Daily Delve: A Shared Daily Dungeon Card Game

Team Project Proposal · October 29, 2026

1. Project title and team members

Title: Daily Delve: A Shared Daily Dungeon Card Game (web application)

Team members:

- Tyler Kazlauskas
- Antonio Ayala
- Aaron Mark
- Justin Morneau

2. Project description and motivation

Daily Delve is a browser-based roguelike card game in which every player worldwide faces the same dungeon each day. At midnight UTC a seed produces that day's map, enemies, and rewards. Players choose a route through the map, fight with a deck of cards, and submit their run. The server verifies the run and ranks everyone on a global leaderboard by how far they got.

What problem does it solve? Roguelike deckbuilders are different on every run, so players cannot compare results or share strategies. Daily puzzle games like Wordle show that a shared daily challenge builds a habit and a community, but few offer real strategic depth. Daily Delve combines the two: a deep, replayable game with a fair, comparable daily challenge.

Who are the intended users? Casual and mid-core players who enjoy short sessions (about 15 to 25 minutes), fans of games like Slay the Spire, and friend groups or classmates who like to compete on a daily board.

Why would someone use it? Everyone plays the same dungeon, so scores are directly comparable and route tips are shareable ("take the left path, the shrine there is worth it"). Each day is a fresh puzzle. Players get one official attempt per day plus unlimited practice runs on the same seed.

3. Main features

Our core features form a playable end-to-end game loop (the MVP). Optional features are added only after the MVP works.

Core feature	What the user does	What the application does
1. Daily seeded dungeon map	Views today's dungeon and chooses a path through a branching map of fights, elites, shrines, and a boss	Derives a seed from the date and generates the same map for every player, with enemies and rewards fixed by the seed
2. Card combat	Draws a hand, spends energy to play cards, reads enemy intents, and picks a new card after each fight	Runs a deterministic turn-based engine with card tags (Fire, Frost, Poison) that create combos, and updates HP, deck, and enemy state
3. Accounts and run submission	Registers, logs in, and plays one official attempt per day (unlimited practice)	Saves each run as the seed plus the player's choices, then re-simulates it on the server to verify the result
4. Global leaderboard	Views today's ranking and their own history	Ranks players by furthest node reached, with ties broken by score, fewer turns, then earlier submission

Optional features (if time permits), in priority order:

- Merchant nodes with a small shop of passive "charms" and seeded rerolls.
- Forge nodes where two cards fuse into a new combined card.
- Daily modifiers that change the rules (for example, Fire cards cost less today).
- AI-generated dungeon names and flavor text, written once per day into a fixed template.
- A replay viewer and a "graveyard map" showing where players died.

4. Application architecture

Front-end. A single-page web app with these screens:

- Login and register.
- Today's Dungeon: the map, where the user selects the next node.
- Combat: hand of cards, energy, player and enemy HP, and visible enemy intents. The user plays cards and ends turns.
- Reward and deck: choose a new card after a fight and view the current deck.
- Run summary: shows the result and submits it.
- Leaderboard and profile: daily rankings and the user's past runs.

Back-end. A server exposing a REST API. Its responsibilities are:

- Authenticating users and managing sessions.
- Serving today's dungeon (seed and metadata).
- Validating all input, including that a submitted run is well-formed.
- Re-simulating submitted runs with the same game engine to confirm the result and prevent cheating.
- Enforcing one official attempt per user per day.
- Computing and returning leaderboard rankings.

Database. Persistent storage for three kinds of data:

- **Users:** username, password hash, display name, join date.
- **Daily dungeons:** date, seed, and (later) modifiers and flavor text.
- **Runs:** user, date, list of choices, furthest node reached, score, turns, whether it was official and verified, and submission time.

The four data operations map to the app as follows. **Create:** registering and submitting runs. **Retrieve:** loading today's dungeon, the leaderboard, and a player's history. **Update:** editing a display name or password and marking a run as verified. **Delete:** removing practice runs or deleting an account.

The game rules live in one shared TypeScript module used by both the browser and the server, so a run always produces the same result in both places.

5. Technology and AI tools

- **Front-end:** TypeScript, React with Vite, HTML and CSS. Cards and the map are drawn with simple SVG and CSS shapes, so no complex art is needed.
- **Back-end:** Node.js with Express (TypeScript), chosen so the server can run the same game engine as the client.

- **Database:** SQLite for development, with PostgreSQL if our hosting requires it.
- **Testing and tooling:** Vitest or Jest for engine tests, and Git with GitHub for version control.
- **Deployment:** A free-tier host such as Render or Railway.
- **AI development tools:** Claude (chat and Claude Code) for planning, scaffolding, writing tests, debugging, and generating placeholder art and card data. Teammates may also use Cursor or Copilot. All AI-written code is reviewed by a teammate and covered by tests before it is merged.

6. Development plan and team responsibilities

Major tasks: game engine and card data, seeded map generation, database and API, authentication, front-end screens, run verification, leaderboard, deployment, testing, and the final demo.

Team member	Initial responsibility
TBD	Game engine, card data, and engine unit tests (shared module)
TBD	Back-end API, database, authentication, and run verification
TBD	Front-end screens: map, combat, and reward flow
TBD	Leaderboard and profile screens, deployment, integration testing, and documentation

Collaboration. We will use one GitHub repository with feature branches and pull requests, and each pull request is reviewed by another teammate. In Week 1 we agree on the API contract (the JSON shapes) so front-end and back-end work can proceed in parallel. We hold a short weekly sync plus chat updates, and merge to the main branch frequently so integration problems surface early.

Milestones (about six weeks):

Week	Milestone
1	Repo and tooling set up, API contract and database design agreed, wireframes drawn, engine skeleton with about 10 cards
2	Combat engine with 25 to 30 cards and unit tests; authentication and database working; static map screen
3	Seeded daily map generation; combat UI connected to the engine; reward flow; first deployment
4	Run submission with server-side verification; leaderboard API and screen. **MVP complete.**
5	Optional features (charm shop first), polish, and bug fixing
6	Final testing, documentation, demo rehearsal, and buffer time

7. Challenges and feasibility

Challenge 1: Keeping the game deterministic. The server can only verify runs if the browser and the server produce identical results. We will put all game rules in one shared module, use a seeded random generator with no clock or unseeded randomness, and write unit tests that replay fixed seeds and compare results.

Challenge 2: Scope creep. A card game can grow endlessly. We will freeze the MVP at four core features, about 25 to 30 cards, and four node types (fight, elite, shrine, boss), and only start optional features after the MVP is working end to end.

Challenge 3: Authentication and integration. Login and database integration are common stumbling blocks. We will use a well-tested library with hashed passwords, build the database and API early (Weeks 2 and 3), and deploy in Week 3 rather than leaving it to the end.

Challenge 4: Coordinating work. Four people changing the same codebase can cause conflicts. We will use the agreed API contract, short-lived branches, reviewed pull requests, and frequent merges.

Feasibility. The MVP uses standard web-app pieces (a React front end, a REST API, and a small relational database) plus a self-contained game engine. It needs no external services, and the art is simple shapes, so the core loop is achievable within the remaining semester, with optional features as a safe buffer.